Universidad de las Américas Puebla

Escuela de ingeniería

Departamento de computación, electrónica y mecatrónica

UDLAP®

Lab report #4

Course: Digital design LRT2022-7

Equipo: 2

183339 Juan Pablo Lopez Moreno LMT
185406 Amy Marianee Ramírez Sánchez LMT

October 20th, 25, San Andrés Cholula, Puebla

1 Abstract

This report will present the results obtained from the forth laboratory practice of the Digital Design course. The main objective of this practice was to begin the development of FPGAs, continuing the use of VHDL as a programming language. The practice focused on the implementation and simulation of combinational logic circuits using both schematic diagrams and Boolean functions.

2 Introduction

Wih the evolution of technologies, digital systems have increasingly incorporated more complex logic behind their functionality, for these reason, technologies have to evolve and improve to suit these needs. For this reason, FPGAs (Field Programmable Gate Arrays) have become a popular choice for implementing digital logic circuits due to their flexibility and reconfigurability. FPGAs allow designers to program custom logic functions directly onto the hardware, enabling rapid prototyping and development of complex digital systems. In order to understand the functioning of the Gate Arrays used, a couple of combinational logic gates will be explored in the lab practice, these gates are as follows:

- Half-substractor, performs the binary subtraction of two single-bit binary numbers. It has two inputs, A and B, and two outputs, Difference and Borrow.
- Multiplexer, combinational circuit that has many data inputs and a single output, depending on control or select inputs.
- Demultiplexer, data distributor combinational circuit. It works in a reverse way of the Multiplexer.
- Magnitud Comparator, combinational circuit that compares two digital or binary numbers in order to find out whether one binary number is equal, less than, or greater than the other binary number.

[1] It's critical to comprehend not only how these combinational circuits operate but also the various ways in which they can be connected within a program or FPGA. Additionally, we can investigate the plethora of possibilities in the field of digital logic by experimenting with various configurations.

3 Objectives

The objectives of this lab are:

- Program and simulate basic combinational logic circuits VHDL.
- Implement basic combination circuits on the Basys 3 board.

For the purpose of this lab, we must understand the following concepts:

- Combinational Logic Circuits

- VHDL Programming
- FPGA Implementation

Of which, we must begin explaining combiantional logic circuits, which are very well-known components in digital electronics, which can provide an output instantly based on the current input. Unlike sequential circuits, a combinational circuit listens for input signals and generates output regardless of the past input or state, as it has no feedback or memory component.[1] VHDL, which has been used in previous labs, is a hardware description language that allows designers to describe the behavior and structure of digital systems. VHDL is widely used for designing FPGAs and ASICs (Application-Specific Integrated Circuits) due to its ability to model complex digital systems at various levels of abstraction.[2] Finally,field programmable gate array (FPGA) is a versatile type of integrated circuit, which, unlike traditional logic devices such as application-specific integrated circuits (ASICs), is designed to be programmable (and often reprogrammable) to suit different purposes, notably high-performance computing (HPC) and prototyping.[3] FPGA implementation involves programming the FPGA device with the designed VHDL code to realize the desired combinational logic circuit. This process includes synthesis, mapping, placement, and routing of the design onto the FPGA fabric.

4 Methodology

The methodology applied in this laboratory practice was structured into three main phases: physical implementation in the laboratory, simulation in Multisim, and programming in EDA Playground.

4.1 Physical Laboratory Phase

During the laboratory work, the following activities were carried out:

- The Boolean function and the truth table corresponding to the schematic shown in Figure 1 were obtained through logical analysis of its structure.
- The schematic and truth table of the Boolean function provided in the practice were determined:
$$F(A, B, C, D) = (B' \cdot D) \cdot (A \oplus B) + (C \cdot A)$$
- The consistency between the derived truth tables and the constructed schematics was verified, ensuring the correct representation of the logical expressions.

4.2 Multisim Simulation Phase

In the Multisim simulation environment, the following procedures were performed:

- Both circuits were virtually constructed using the logic components available in the software.
- Simulations were executed to validate the expected behavior of each circuit.
- The obtained results were compared with the analytically derived truth tables to confirm the correctness of the designs.

4.3 EDA Playground Programming Phase

Finally, in the EDA Playground platform, the following actions were conducted:

- Both Boolean functions were programmed using the VHDL language, following proper coding conventions.
- Testbenches were created and applied to verify the correct functionality of the circuits under all possible input combinations.
- The simulation results were validated against those obtained in the previous phases.

5 Results

In this lab, the physical circuits were not built, but based on certain truth tables, EDAWaves were generated in order to accomplish every single combinational circuit given.:

5.1 Half-substractor

For the half-substractor, the following truth table was used to generate the EDA Code:

Input		**Output**	
A	**B**	**AC**	**R**
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Table 1: Truth Table for the Half-substractor

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 1: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_half is
port(
    A : in std_logic;       -- OR gate input
    B : in std_logic;
    AC : out std_logic;
    R : out std_logic);     -- OR gate output

end ent_half;
```

```
-- Architecture definition

architecture arq_half of ent_half is

 begin

    AC <= (not(A)and(B));
    R <= ((not(A)and(B))or((A)and(not(B))));

end arq_half;
```

With it's appropriate testbench:

Código 2: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_half is
port(
        A : in std_logic;       -- OR gate input
    B : in std_logic;
    AC : out std_logic;
    R : out std_logic);
end component;

signal a_in, b_in, ac_out, r_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_half port map(a_in, b_in, ac_out, r_out);

  process
  begin

    a_in <= '0';
    b_in <= '0';
    wait for 1 ns;
```

```
        a_in <= '0';
    b_in <= '1';
    wait for 1 ns;

    a_in <= '1';
    b_in <= '0';
    wait for 1 ns;
    a_in <= '1';
    b_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

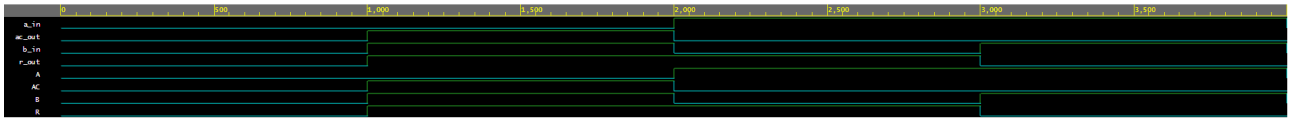

Figure 1: Wave generated for the Half-substractor.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

Figure 2: Evidence of the functioning of the Half-substractor in the Basys 3 board.

5.2 Multiplexor

For the multiplexor, the following truth table was used to generate the EDA Code:

Input			Output
Se	I_1	I_0	**out**
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Table 2: Truth Table for the 2-to-1 Multiplexor

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 3: Code 1: For exercise 2

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_mult is
port(
    SE : in std_logic;        -- OR gate input
    I1 : in std_logic;
    I0 : in std_logic;
    Y : out std_logic);     -- OR gate output

end ent_mult;

-- Architecture definition

architecture arq_mult of ent_mult is

 begin

    Y <= ((not(SE)and(I0)))or((SE)and(I1));

end arq_mult;
```

With it's appropriate testbench:

Código 4: Code 2: Testbench for exercise 2

```
library IEEE;
use IEEE.std_logic_1164.all;
```

```
entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_mult is
port(
    SE : in std_logic;       -- OR gate input
    I1 : in std_logic;
    I0 : in std_logic;
    Y : out std_logic);
end component;

signal se_in, i1_in, i0_in, y_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_mult port map(se_in, i1_in, i0_in, y_out);

  process
  begin

    se_in <= '0';
    i1_in <= '0';
    i0_in <= '0';
    wait for 1 ns;

        se_in <= '0';
    i1_in <= '0';
    i0_in <= '1';
    wait for 1 ns;

    se_in <= '0';
    i1_in <= '1';
    i0_in <= '0';
    wait for 1 ns;

    se_in <= '0';
    i1_in <= '1';
    i0_in <= '1';
    wait for 1 ns;
```

```
        se_in <= '1';
        i1_in <= '0';
        i0_in <= '0';
        wait for 1 ns;

        se_in <= '1';
        i1_in <= '0';
        i0_in <= '1';
        wait for 1 ns;

        se_in <= '1';
        i1_in <= '1';
        i0_in <= '0';
        wait for 1 ns;

        se_in <= '1';
        i1_in <= '1';
        i0_in <= '1';
        wait for 1 ns;

    end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

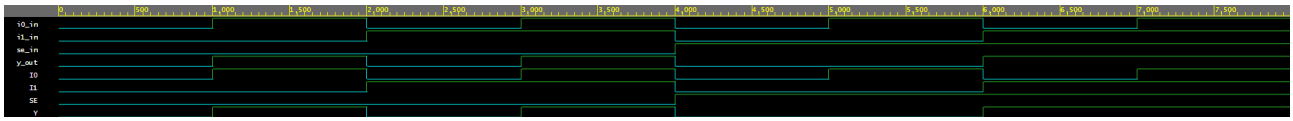

Figure 3: Wave generated for the Multiplexor.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

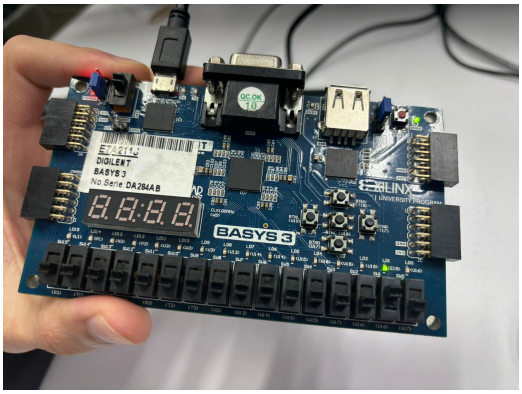

Figure 4: Evidence of the functioning of the Multiplexor in the Basys 3 board.

5.3 Demultiplexor

For the demultiplexor, the following truth table was used to generate the EDA Code:

Input		**Output**	
Se	I_0	O_1	O_0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0

Table 3: Truth Table for the 1-to-2 Demultiplexor

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 5: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_demult is
port(
    SE : in std_logic;        -- OR gate input
    I0 : in std_logic;
    O1 : out std_logic;
    O0 : out std_logic);      -- OR gate output

end ent_demult;

-- Architecture definition

architecture arq_demult of ent_demult is

 begin

    O1 <= (((SE)and(I0)));
    O0 <= ((not(SE)and(I0)));


end arq_demult;
```

With it's appropriate testbench:

Código 6: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;
```

```
entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_demult is
port(
    SE : in std_logic;       -- OR gate input
    I0 : in std_logic;
    O1 : out std_logic;
    O0 : out std_logic);     -- OR gate output

end component;

signal se_in, i0_in, O1_out, O0_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_demult port map(se_in, i0_in, O1_out, O0_out);

  process
  begin

    se_in <= '0';
    i0_in <= '0';
    wait for 1 ns;

        se_in <= '0';
    i0_in <= '1';
    wait for 1 ns;

    se_in <= '1';
    i0_in <= '0';
    wait for 1 ns;

    se_in <= '1';
    i0_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

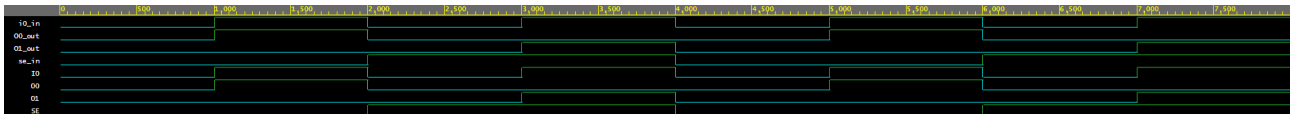

Figure 5: Wave generated for the Demultiplexor.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

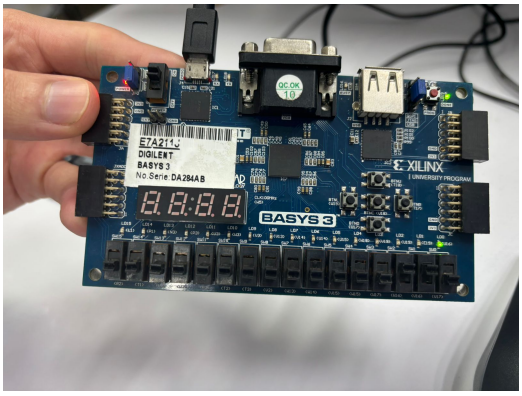

Figure 6: Evidence of the functioning of the Demultiplexor in the Basys 3 board.

5.4 Magnitud Comparator

For the comparator, the following truth table was used to generate the EDA Code:

Input		**Output**		
A	**B**	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Table 4: Truth Table for a 1-bit Magnitude Comparator

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 7: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration
```

```
entity ent_comp is
port(
    A : in std_logic;       -- OR gate input
    B : in std_logic;
    IG : out std_logic;
    MI : out std_logic;
    MA : out std_logic);     -- OR gate output

end ent_comp;

-- Architecture definition

architecture arq_comp of ent_comp is

 begin

    MA <= (A)and(not(B));
    IG <= ((not(A))and(not(B)))or((A)and(B));
    MI <= (B)and(not(A));

end arq_comp;
```

With it's appropriate testbench:

Código 8: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_comp is
port(
    A : in std_logic;       -- OR gate input
    B : in std_logic;
    IG : out std_logic;
    MI : out std_logic;
    MA : out std_logic);     -- OR gate output
end component;

signal a_in, b_in, ig_out, mi_out, ma_out: std_logic;
```

```
begin

  -- Connect UUT
  UUT: ent_comp port map(a_in, b_in, ig_out, mi_out, ma_out);

  process
  begin

    a_in <= '0';
    b_in <= '0';
    wait for 1 ns;

        a_in <= '0';
    b_in <= '1';
    wait for 1 ns;

    a_in <= '1';
    b_in <= '0';
    wait for 1 ns;

    a_in <= '1';
    b_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

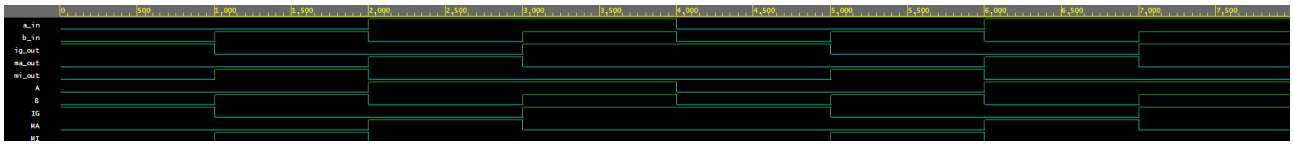

Figure 7: Wave generated for the Magnitud Comparator.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

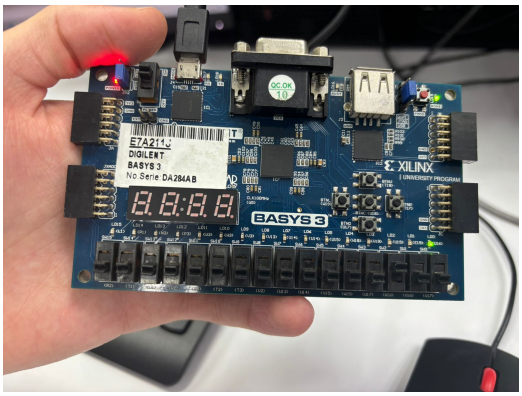

Figure 8: Evidence of the functioning of the Magnitud Comparator in the Basys 3 board.

6 Conclusions

The development of this practice allowed the fulfillment of the laboratory objectives, as Boolean functions were obtained from schematics, and schematics were interpreted from Boolean functions. Additionally, the Boolean functions were programmed and simulated using VHDL, confirming their correct logical behavior. The detailed identification of gates, analysis of partial outputs, and the creation of truth tables and canonical forms allowed algebraic validation of the results. Simulations performed in Multisim and EDA Playground corroborated the consistency between theory and practice, demonstrating the usefulness of digital simulation tools for predicting and verifying circuit operation. In this way, the practice not only enabled experimental validation of theoretical principles but also reinforced the understanding of programming and simulation of Boolean functions in digital environments.

References

[1] GeeksforGeeks. (2025) What is combinational circuit ? [En línea]. Disponible: https://www.geeksforgeeks.org/digital-logic/what-is-combinational-circuit

[2] Intel Corporation, "Vhdl definition," https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm, 2017, accessed: 2024-10-27.

[3] J. Schneider and I. Smalley. (2025) Field programmable gate array. [En línea]. Disponible: https://www.ibm.com/think/topics/field-programmable-gate-arrays